第38课 动态规划入门（斐波那契、爬楼梯）

本课为选学内容，听不懂没关系，以后还会再学！

一、课程信息

项目	内容
课次	第 38 课 / 共 40 课
主题	动态规划入门（斐波那契、爬楼梯）——**选学，听不懂可跳过**
前置知识	递归（第29-30课）、数组（第10-11课）、函数（第13课）
后续衔接	第39课 动态规划经典题（选学）

教学目标（可检验）

1. **能用自己的话说出**动态规划的核心思想：把大问题拆成小问题，用数组记住算过的答案，避免重复计算。
2. **能写出**斐波那契数列的 DP（数组递推）版本，并说出它比递归版快在哪里。
3. **能独立完成**爬楼梯问题，写出 `dp[i] = dp[i-1] + dp[i-2]` 的递推代码。

二、教学重点与难点

重点

- 动态规划的核心思想："记住算过的答案，不重复算"。
- 从递归到 DP 的演进：递归版斐波那契（慢） DP 版斐波那契（快）。
- 爬楼梯问题的状态转移方程：`dp[i] = dp[i-1] + dp[i-2]`。

难点

- 理解"为什么递归版斐波那契会慢"——重复计算的直观感受。
- "状态转移方程"的概念——听起来很高级，其实就是"大问题怎么由小问题拼起来"。
- 从递归思维转换到"填表格"思维——不是从大往小拆，而是从小到大一步步算。

> **教师提醒**：本课是选学内容。教学中应反复强调"听不懂没关系"，让学生放松心态。只要能感受到 DP 的思想就算成功，不要求所有学生都能独立写出代码。

三、课堂时间分配（共 100 分钟）

时间段	时长	环节	内容	形式
0′–5′	5 min	导入	"备忘录"故事引入，回顾递归版斐波那契的问题	讲述 + 互动
5′–25′	20 min	讲解①	DP 核心思想 + 斐波那契数列：递归版 vs DP 版	讲解 + 板书画图
25′–40′	15 min	练习①	学生上机写斐波那契 DP 版，对比递归版速度	上机实操
40′–45′	5 min	休息/鼓励	表扬进度，强调"听不懂也很正常"	互动
45′–65′	20 min	讲解②	爬楼梯问题（生活引入分析代码状态转移方程概念）	讲解 + 板书画图
65′–80′	15 min	练习②	课堂练习题上机	上机实操
80′–90′	10 min	例题③	变式：一次走1、2或3级台阶	讲解 + 互动
90′–97′	7 min	总结	回顾 DP 核心、常见错误、鼓励学生	互动
97′–100′	3 min	布置	课后作业 + 下节课预告	讲述

> **说明**：两段主要讲解分别为 20 分钟，均不超过 25 分钟上限，中间穿插大量动手练习和鼓励环节。

四、讲稿提纲

4.1 导入——"备忘录"的故事（5 min）

> 同学们好！在开始之前，老师先告诉大家——**今天的内容是选学的！** 什么意思呢？就是说，如果你能听懂，那非常棒；如果觉得有点难，也完全没关系！这个知识点以后还会再学很多遍。今天我们只要"认识一下"就好，不用紧张哦！
>
> 好，我们先来讲一个故事。
>
> 小明的妈妈让他去超市买东西，给了他一张**购物清单**。小明到了超市，买了牛奶，回家了。妈妈又说："哎，再去买个面包。"小明又跑了一趟超市。妈妈又说："再去买个鸡蛋。"小明又跑了一趟……
>
> 小明跑了三趟超市，累坏了！
>
> 如果一开始妈妈把所有东西都写在**一张纸条**上，小明跑一趟就能全买回来，是不是聪明多了？
>
> 在编程里也有类似的情况。还记得我们之前学过的斐波那契数列递归吗？

（在黑板上写出斐波那契数列：1, 1, 2, 3, 5, 8, 13, 21, …）

> 我们当时用递归写了 `fib(n) = fib(n-1) + fib(n-2)`。
>
> 但是有同学试过 `fib(40)` 吗？**超——级——慢！** 为什么呢？因为电脑在**反复做同样的事情**，就像小明反复跑超市一样！
>
> 今天我们要学一个聪明办法——**动态规划**。它的核心就一句话：**算过的就记下来，下次直接查！** 就像写了一张**备忘录**。

4.2 回顾递归版斐波那契——它为什么慢？（5 min，板书核心）

> 我们来看看 `fib(5)` 的递归调用过程：

（在黑板上画树形图）

```
            fib(5)
           /      \
        fib(4)    fib(3)
        /    \     /    \
     fib(3) fib(2) fib(2) fib(1)
     /    \
  fib(2) fib(1)
```

> 大家数一数，`fib(3)` 被算了几次？——**2 次！**
> `fib(2)` 被算了几次？——**3 次！**
> 才算到 fib(5) 就重复这么多了。如果算 fib(40)，fib(2) 要被重复算**几千万次**！难怪电脑慢到爆炸。
> 问题出在哪里？—— **电脑没有"记忆力"！** 它算完 fib(3) = 2 之后就忘了，下次遇到 fib(3) 又得重新算一遍。
> 那怎么办？**给电脑一个备忘录呗！** 用一个**数组**把每个 fib(i) 的答案存起来，下次要用直接查！

4.3 DP 的核心思想（5 min）

> 动态规划，英文叫 Dynamic Programming，简称 **DP**。名字听起来很高级，但核心思想其实很简单：

板书（大字写）：

```
DP = 备忘录 = 算过的就记下来，下次直接查！
```

> 用生活里的例子来说：
> **计算器上的 M+ 键**：你算出一个中间结果，按 M+ 存起来，以后随时按 MR 调出来用。DP 就是这个意思！
> **考试小抄**（当然考试不能用哈）：你把重要的公式记在纸上，考试的时候直接看，不用重新推导。DP 就是让电脑"写小抄"！
> **搭积木**：想盖10层楼？先盖第1层，再盖第2层……每一层都"站在前面楼层的肩膀上"。DP 也是从小问题一步步推到大问题！
> DP 分三步走：
> 1. **找出小问题和大问题的关系**（公式 / 递推关系）
> 2. **从最小的问题开始算**，把答案存到数组里
> 3. **一步步算到大问题**，每次都用已经算好的答案
>
> 对比递归：递归是"从大到小拆"，DP 是"从小到大填"。效果一样，但 DP **不重复算**，所以快得多！

4.4 斐波那契数列——从递归到 DP（10 min，讲解①核心）

> 好，我们来动手改造斐波那契！

第一版：递归（旧方法，慢）

```
int fib(int n) {
    if (n == 1 || n == 2) return 1;
    return fib(n - 1) + fib(n - 2);  // 大量重复计算！
}
```

> 这个大家学过了。问题是：**重复计算太多，算 fib(40) 要等很久。**

第二版：DP（新方法，快！）

> 换个思路：我们准备一个数组 `dp[]`，从小到大把每个斐波那契数**存起来**。

```
dp[1] = 1          （第1个数是1）
dp[2] = 1          （第2个数是1）
dp[3] = dp[2] + dp[1] = 1 + 1 = 2
dp[4] = dp[3] + dp[2] = 2 + 1 = 3
dp[5] = dp[4] + dp[3] = 3 + 2 = 5
......
```

板书：手动填表

```
i:      1   2   3   4   5   6   7   8
dp[i]:  1   1   2   3   5   8  13  21
```

> 看！我们只需要一个 for 循环，从前往后算，每次查一下前两个已经存好的数就行了。
> **没有任何重复计算！**
> 这就像填一张表格：先填第 1 格、第 2 格，然后每次看看前两格填什么，把它们加起来填到当前格。
> 简简单单！

4.5 鼓励过渡（5 min，休息 + 鼓励）

> 同学们，到目前为止，你们已经理解了 DP 最核心的思想了！给自己鼓个掌！
> 接下来我们再看一个**超级经典**的 DP 问题——爬楼梯。这个问题在竞赛和面试中**特别常见**，学会了可以吹很久的牛哦！
> 如果前面的内容有点懵，也完全没关系。跟着老师一起看，能理解多少就理解多少。

4.6 爬楼梯问题（20 min，讲解②核心）

> **故事引入：**
> 小华家住在 5 楼（没有电梯）。他每次爬楼梯，一步可以走 **1 级** 或者 **2 级**。
> 问：从地面走到第 5 级台阶，一共有多少种不同的走法？
> 先来试试简单的情况：

板书：手动列举

```
第 1 级台阶：只有 1 种走法
   [1]

第 2 级台阶：有 2 种走法
   [1, 1]
   [2]

第 3 级台阶：有 3 种走法
   [1, 1, 1]
   [1, 2]
   [2, 1]
```

> 第 4 级呢？你可以继续列举，但越来越多了……有没有更聪明的方法？

引导学生发现规律：

> 关键问题：**你站在第 4 级台阶上，你是怎么到这里的？**
> 只有两种可能：
> 1. 你是从**第 3 级**走了 1 步上来的
> 2. 你是从**第 2 级**跨了 2 步上来的
>
> 所以，**到第 4 级的走法 = 到第 3 级的走法 + 到第 2 级的走法！**
> 用公式写就是：

板书（大字写，这是本课最重要的公式）：

```
dp[i] = dp[i-1] + dp[i-2]
```

> 这个公式有个正式的名字，叫**状态转移方程**。名字听着高级，其实意思很简单——
> "**当前状态**（走到第 i 级的方法数）= **由哪些小状态转移过来**（第 i-1 级 + 第 i-2 级）"
> 和斐波那契数列的公式一模一样！爬楼梯本质上就是一个斐波那契问题！

板书：填表

```
i:      1   2   3   4   5   6   7
dp[i]:  1   2   3   5   8  13  21

验证 dp[4] = dp[3] + dp[2] = 3 + 2 = 5
验证 dp[5] = dp[4] + dp[3] = 5 + 3 = 8
```

> 现在回答开头的问题：走到第 5 级台阶有 **8** 种走法！

4.7 状态转移方程——简单提及

> 刚才我们用到的公式 `dp[i] = dp[i-1] + dp[i-2]` 叫做**状态转移方程**。
> 你们不用背这个名字，只需要记住意思：
> **状态转移方程 = "大问题怎么由小问题拼起来"的公式**
> 每道 DP 题目最关键的一步，就是找到这个公式。找到了，代码就非常好写。
> 以后遇到 DP 问题，就问自己一句话："**我现在这个状态，能从哪些更小的状态推过来？**"

七、常见错误与应对

错误 1：dp 数组初始值搞错

```
//  错误：爬楼梯 dp[2] 应该是 2，不是 1！
dp[1] = 1;
dp[2] = 1;  // 错！第2级台阶有2种走法（1+1 或 2）
```

现象：答案全都偏小。

应对：

> "写 DP 第一步，先想想最小的情况答案是什么。第 1 级有几种走法？1 种。第 2 级呢？2 种。用小数据手动验证！"
> **注意区分**：斐波那契数列的 dp[2] = 1（因为第2个斐波那契数就是1），但爬楼梯的 dp[2] = 2（因为走到第2级有两种走法）。虽然公式一样，但初始值不同！

错误 2：for 循环从 1 开始，覆盖了初始值

```
//  错误：循环从 i=1 开始，把 dp[1] 和 dp[2] 都覆盖了！
dp[1] = 1;
dp[2] = 2;
for (int i = 1; i <= n; i++) {  // 应该从 i=3 开始！
    dp[i] = dp[i - 1] + dp[i - 2];
}
```

现象：dp[1] 和 dp[2] 被错误覆盖，并且 dp[1] 会访问 dp[-1] 导致越界。

应对：

> "初始值是 DP 的'地基'，不能被 for 循环覆盖！for 循环要从初始值的**下一个**开始。"

错误 3：没用 long long，大数据溢出

```
//  错误：n=50 时，dp[50] 已经超过 int 范围
int dp[55];  // 应该用 long long！
```

现象：n 较大时输出负数或奇怪的数字。

应对：

> "斐波那契数列增长非常快，第 46 项就超过 int 的上限了。养成好习惯——DP 数组用 `long long`！"

错误 4：把递归和 DP 搞混，在 DP 里又写了递归

```
//  错误：这不是 DP，还是递归!
long long dp(int n) {
    if (n == 1 || n == 2) return 1;
    return dp(n - 1) + dp(n - 2);
}
```

现象：虽然函数名叫 dp，但其实还是递归，还是会很慢。

应对：

> "DP 的标志是**用数组存答案** + **for 循环从小到大填**。如果你发现自己在用递归调用，那就不是 DP，只是换了个名字的递归。记住：DP = 数组 + for 循环！"

错误 5：忘记处理 n=1 或 n=2 的特殊情况

```
//  当 n=1 时，循环不执行，dp[1] 正确
// 但如果代码里写了 dp[2] = 2，而 n=1，访问 dp[2] 没问题（只是多写了一步）
// 真正的坑是：如果 n < 3，for 循环不执行，直接输出 dp[n] 是否正确?
```

应对：

> "写完代码后，一定要用 n=1 和 n=2 这两个最小值测试一下。如果答案对了，大数据一般就没问题！"

九、下节课预告

> 同学们，今天我们认识了一位新朋友——**动态规划（DP）**！
>
> 我们学到了 DP 的核心思想：**算过的就记下来，不重复算**。用一个数组当"备忘录"，从小到大一步步推出答案。
>
> 我再说一次：**今天的内容是选学的！** 如果你完全听懂了，那你真的很棒，已经开始接触竞赛中最重要的算法之一了！如果觉得有点云里雾里，也完全没关系——DP 是一个需要**反复学习、大量练题**才能掌握的技能，很多初中生甚至高中生也在学。你现在能有一个初步的印象，就已经走在了同龄人的前面！
>
> **下节课——第39课《动态规划经典题》**（也是选学），我们会看更多有趣的 DP 问题：
>
> - 数字三角形：从三角形顶部走到底部，怎么走才能让经过的数字之和最大？
> - 更多 DP 的经典应用
>
> 回去好好消化今天的内容，把爬楼梯的代码多写几遍，手动填填表，体会"从小到大推"的感觉。**做不出作业也不要有压力**，带着问题来上课就好！下节课见！

> **教师备注**：
>
> 1. 本课是选学内容，教学过程中应反复给学生减压。建议在课堂上多说"听不懂没关系""以后还会学""你们已经很棒了"等鼓励性语言。
> 2. 重点是让学生**感受** DP 的思想（用数组记住答案避免重复计算），而非要求所有学生都能独立写出代码。
> 3. 斐波那契递归版 vs DP 版的速度对比是本课最直观的"aha moment"，建议让学生亲手运行两个程序，感受差距。
> 4. 爬楼梯问题的状态转移方程 `dp[i] = dp[i-1] + dp[i-2]` 是核心，务必在黑板上手动填表演示，让学生跟着一起填。
> 5. 练习4和作业2是给学有余力的学生准备的挑战，不强制要求完成。

教案结束